

Relazione Individuale Progetto

Corso di Web Applications (A.A. 2025/26) – Progetto *Wikiwelp*

Matricola: 250512 Cognome: Ricca Nome: Giovanni

1. Attività Svolte personalmente

Nel progetto ho strutturato lo sviluppo dell'architettura Backend e l'integrazione con il Frontend usando Angular:

- **Lato Backend (Spring Boot & Postgres):**
 - Modellazione del database **Postgres** su **5 tabelle**: `users`, `pages`, `tags`, `page_revisions` e la tabella associativa `page_tags`.
 - Implementazione del **pattern DAO** per l'accesso al database Postgres (`PageDao`, `UserDao`, `TagDao`, `RevisionDao`, `PageTagDao`) e gestione connessioni con `DBManager`.
 - Implementazione del **pattern Proxy** (`PageProxyDTO`) su relazioni 1:N (`page_revisions`) e N:M (`page_tags`) per il caricamento su richiesta.
 - Implementazione dei **@RestController** per la gestione delle richieste da Angular, con scambio dati in formato **JSON** e gestione degli errori SQL.
- **Lato Frontend (Angular):**
 - Costruzione del servizio **ServizioService** per la gestione multi-utente (sessioni, check privilegi admin).
 - Opportune chiamate `HttpClient` sul Backend per integrazione frontend (home, edit, admin, login, ...).
 - Integrazione dell'API di Wikipedia come fallback se una voce non è presente nel database Postgres.
 - Integrazione della libreria esterna **Syncfusion Rich Text Editor** in `edit.component.ts` per abilitare la formattazione WYSIWYG e il caricamento delle immagini.
- **Supporto agli altri membri:**
 - Setup dell'ambiente di sviluppo e supporto a Romualdo Tomaselli nell'interfacciamento tra template HTML/CSS dell'interfaccia responsive e codice Angular.

2. Collaborazione con il gruppo

- **Coordinamento:** Gestione del codice su repository GitHub con branch dedicati e brevi allineamenti periodici sullo stato di avanzamento.
- **Decisioni comuni:** Scelta congiunta dello stack (Spring Boot, Postgres, Angular), strutturazione delle 5 tabelle del DB e specifica dei contratti JSON dei **@RestController**.
- **Supporto e revisione del lavoro altrui:** Code review periodiche con opportune ripetizioni per verificare l'integrazione e il corretto funzionamento complessivo.

3. Problemi affrontati e soluzioni

- **Setup ambiente e testing veloce:** Risolto dockerizzando l'applicazione tramite Docker Compose, garantendo avvio immediato, riproducibilità e collaudo rapido tra i membri del gruppo.
- **Salvataggio delle immagini nell'editor:** Risolto gestendo le immagini tramite codifica in **Base64** all'interno del contenuto, permettendo il salvataggio diretto nel database Postgres senza storage dedicati.
- **Pattern Proxy sulle relazioni 1:N e N:M:** Risolto estendendo `PageDTO` con `PageProxyDTO`, posticipando l'invocazione di `PageTagDao` e `RevisionDao` alla chiamata dei rispettivi getter, ottimizzando le query su Postgres.
- **Web App multi-utente e persistenza login:** Risolto centralizzando la gestione dello stato e dei permessi di amministratore nel `ServizioService`, sincronizzandoli con il `localStorage`.
- **Cosa si sarebbe potuto fare diversamente:** Il salvataggio e il check della password degli utenti avrebbero potuto fare uso di un hashing (es. algoritmo SHA) invece di memorizzarla e verificarla in chiaro nel database.

4. Impegno e partecipazione

- **Partecipazione:** Costante, continuativa e proattiva per tutta la durata del progetto.
- **Rispetto delle scadenze:** Rispetto puntuale di tutte le scadenze, completando l'architettura backend (DAO, Proxy, @RestController) e la logica Angular in tempo per il collaudo della UI responsive.
- **Criticità riscontrate:** Discrepanze iniziali sui formati JSON scambiati tra frontend e backend, risolte tempestivamente definendo e condividendo i contratti DTO.